

Logical Foundations for Concurrency: Non-Determinism and Failures

Jorge A. Pérez
University of Groningen, The Netherlands
j.a.perez [[at]] rug.nl

ECI 2026
(Version of 31st July 2026)

Outline

Context

Logical Foundations for Concurrency

Curry-Howard correspondence for Concurrency

Example: Movie Server

Adding Non Determinism and Failure

Syntax and Semantics

Types

Properties

Examples

Extended Movie Server

Encoding λ^{exc}

Concluding Remarks

Preamble

- ▶ This presentation is based on joint work with Luís Caires (Lisbon).
- ▶ The paper was originally presented at ESOP 2017 (European Symposium on Programming Language and Systems).
- ▶ While it has been a while, over the years the ideas in this work have proven influential in several ways. In particular:
 - ▶ New encodings of λ -calculi with **intersection types** as session-typed processes (jww Daniele Nantes and Joe Paulus)
 - ▶ New process languages with **shared state**, implemented as the **CLASS programming language** (Pedro Rocha and Luís Caires)
- ▶ Many exciting things to do!

Outline

Context

Logical Foundations for Concurrency

- Curry-Howard correspondence for Concurrency

- Example: Movie Server

Adding Non Determinism and Failure

- Syntax and Semantics

- Types

- Properties

Examples

- Extended Movie Server

- Encoding λ^{exc}

Concluding Remarks

Behavioral Types (1/2)

By classifying **values**, traditional data types are an effective basis for validating sequential programs

To reason about services, **behavioral types** classify **interactions**

- High-level representations of communication structures
- Compositional tools for (statically) checking service behavior
- Tied to programming abstractions that promote communication as a first-class concern

Behavioral Types (2/2)

- Typically developed upon **process calculi**
 - ★ Variants of the π -calculus
 - ★ Expressive core programming models; suited for investigation
- Formal specification languages, based on communication
 - ★ Strong ties with automata, logic, type theory
 - ★ Clear linkage with validation methods
 - ★ Precise notions of runtime correctness
- General verification techniques tailored to actual languages (Java, Scala, ML, Haskell). Many recent advances.
- A notable class: **session types**

Session Types (1/2)

Seminal type-based approach to the analysis of structured communications [Honda, Vasconcelos, Kubo (1998)]

- Communication protocols structured into **sessions**
- Concurrent processes interacting along **session channels**
- Disciplined interactive behavior, abstracted as **session types**

Session Types (2/2)

Session specifications usually given as π -calculus processes

- Actions always occur in **dual** pairs
- New sessions created by invoking **shared servers**
- **Concurrency** in the simultaneous execution of sessions
- **Mobility** in the exchange of session and server names

Correctness Guarantees for Specifications

- Adhere to their ascribed session protocols - **Fidelity**
- Do not feature runtime errors – **Safety**
- Do not get stuck – **Progress / Lock-Freedom**
- Do not have infinite reduction sequences – **Termination**

Session Types (2/2)

Session specifications usually given as π -calculus processes

- Actions always occur in **dual** pairs
- New sessions created by invoking **shared servers**
- **Concurrency** in the simultaneous execution of sessions
- **Mobility** in the exchange of session and server names

Correctness Guarantees for Specifications

- Adhere to their ascribed session protocols - **Fidelity**
- Do not feature runtime errors – **Safety**
- Do not get stuck – **Progress / Lock-Freedom**
- Do not have infinite reduction sequences – **Termination**

In this talk: **binary** session types (two-party protocols).

Outline

Context

Logical Foundations for Concurrency

Curry-Howard correspondence for Concurrency

Example: Movie Server

Adding Non Determinism and Failure

Syntax and Semantics

Types

Properties

Examples

Extended Movie Server

Encoding λ^{exc}

Concluding Remarks

Logical Foundations for Session Types

Curry-Howard correspondence for Concurrency:

propositions	$\leftrightarrow$	session types
sequent proofs	$\leftrightarrow$	π -calculus processes
cut elimination	$\leftrightarrow$	process communication

Developed by Caires & Pfenning for intuitionistic linear logic (2010).
Adapted to classical linear logic by Wadler (2012).

Logical Foundations for Session Types

Curry-Howard correspondence for Concurrency:

propositions	$\leftrightarrow$	session types
sequent proofs	$\leftrightarrow$	π -calculus processes
cut elimination	$\leftrightarrow$	process communication

Developed by Caires & Pfenning for intuitionistic linear logic (2010).
Adapted to classical linear logic by Wadler (2012).

Main Features

- Clear account of resource usage policies in concurrency
- Session fidelity, runtime safety, global progress “for free”
- Excellent basis for generalizations and extensions

Process Model: Syntax

$P, Q ::= \bar{x}(y).P \mid x(y).P \mid !x(y).P$	[output and (replicated) input]
$\mid P \mid Q \mid (\nu y)P \mid 0$	[parallel, restriction, inaction]
$\mid x.\text{case}(P, Q)$	[branching]
$\mid x.\text{inr}; P \mid x.\text{inl}; P$	[right and left selection]
$\mid x.\overline{\text{close}} \mid x.\text{close}; P$	[session termination]
$\mid [x \leftrightarrow y]$	[channel forwarder]

Process Model: Reduction Semantics [Excerpt]

Reduction ($\rightarrow$) plus structural congruence ($\equiv$):

$$\begin{aligned}\bar{x}(y).Q \mid x(y).P &\rightarrow (y)(Q \mid P) \\ \bar{x}(y).Q \mid !x(y).P &\rightarrow (y)(Q \mid P) \mid !x(y).P \\ x.\text{inl}; P \mid x.\text{case}(Q, R) &\rightarrow P \mid Q \\ x.\overline{\text{close}} \mid x.\text{close}; P &\rightarrow P \\ (x)([x \leftrightarrow y] \mid P) &\rightarrow P\{y/x\} \\ P \equiv P', P' \rightarrow Q', Q' \equiv Q &\Rightarrow P \rightarrow Q\end{aligned}$$

Types and Duality (CLL)

Types are assigned to names and describe their session behavior:

$x : A \otimes B$	[Output name of type A , continue as B]
$x : A \wp B$	[Input name of type A , continue as B]
$x : !A$	[Persistent offer of A]
$x : ?A$	[Request of a persistent behavior A]
$x : A \& B$	[Offer <i>both</i> A and B]
$x : A \oplus B$	[Select <i>either</i> A or B]
$x : 1$	[Terminated interaction]
$x : \perp$	[Terminated interaction]

Duality of A , written $\overline{A}$, is standard.

Type Judgments

$$P \vdash \underbrace{x_1:A_1, \dots, x_k:A_k}_{\Delta : \text{linear services}} ; \underbrace{u_1:B_1, \dots, u_n:B_n}_{\Theta : \text{shared services}}$$

- Judgments correspond to logical sequents in the classical linear logic Σ_2 [Andreoli'92]
- Supports exponentials $!A$ and $?A$ in terms of standard semantics for lazy replication in the π -calculus

Sample Typing Rules: Identity, Mix & Cut

$$\overline{\vdash A, \overline{A}; \Theta} \text{ (Tid)}$$

$$\frac{\vdash \Delta; \Theta \quad \vdash \Delta'; \Theta}{\vdash \Delta, \Delta'; \Theta} \text{ (T |)}$$

$$\frac{\vdash \Delta, A; \Theta \quad \vdash \Delta', \overline{A}; \Theta}{\vdash \Delta, \Delta'; \Theta} \text{ (Tcut)}$$

Sample Typing Rules: Identity, Mix & Cut

$$\frac{}{[x \leftrightarrow y] \vdash x:A, y:\overline{A}; \Theta} (\text{Tid})$$

$$\frac{P \vdash \Delta; \Theta \quad Q \vdash \Delta'; \Theta}{P \mid Q \vdash \Delta, \Delta'; \Theta} (\text{T} \mid)$$

$$\frac{P \vdash \Delta, x:A; \Theta \quad Q \vdash \Delta', x:\overline{A}; \Theta}{(x)(P \mid Q) \vdash \Delta, \Delta'; \Theta} (\text{Tcut})$$

Sample Typing Rules: Input & Output

$$\frac{P \vdash \Delta, y:A; \Theta \quad Q \vdash \Delta', x:B; \Theta}{\overline{x}(y).(P \mid Q) \vdash \Delta, \Delta', x:A \otimes B; \Theta} (\text{T}\otimes)$$

$$\frac{P \vdash \Delta, y:C, x:D; \Theta}{x(y).P \vdash \Delta, x:C \wp D; \Theta} (\text{T}\wp)$$

Sample Typing Rules: Input & Output

$$\frac{P \vdash \Delta, y:A; \Theta \quad Q \vdash \Delta', x:B; \Theta}{\overline{x}(y).(P \mid Q) \vdash \Delta, \Delta', x:A \otimes B; \Theta} (\text{T}\otimes)$$

$$\frac{P \vdash \Delta, y:C, x:D; \Theta}{x(y).P \vdash \Delta, x:C \wp D; \Theta} (\text{T}\wp)$$

Reduction:

$$(x)(\overline{x}(y).(P \mid Q) \mid x(y).R) \rightarrow (x)(y)(P \mid Q \mid R)$$

[Free output $x\langle y \rangle.P$ representable as $\overline{x}(z).(P \mid [z \leftrightarrow y]).$]

Sample Typing Rules: Exponentials

$$\frac{\vdash A; \Theta}{\vdash !A; \Theta}(\mathsf{T}!) \quad \frac{\vdash \Delta, A; A, \Theta}{\vdash \Delta; A, \Theta}(\mathsf{T}_{\text{copy}})$$

$$\frac{\vdash \Delta; A, \Theta}{\vdash \Delta, ?A; \Theta}(\mathsf{T}?) \quad \frac{\vdash \bar{A}; \Theta \quad \vdash \Delta; A, \Theta}{\vdash \Delta; \Theta}(\mathsf{T}_{\text{cut}}?)$$

Sample Typing Rules: Exponentials

$$\frac{P \vdash y:A; \Theta}{!x(y).P \vdash x:!A; \Theta} (\text{T}!) \quad \frac{P \vdash \Delta, y:A; x:A, \Theta}{\bar{x}(y).P \vdash \Delta; x:A, \Theta} (\text{T}_{\text{copy}})$$

$$\frac{P \vdash \Delta; x:A, \Theta}{P \vdash \Delta, x: ?A; \Theta} (\text{T}?) \quad \frac{P \vdash y:\bar{A}; \Theta \quad Q \vdash \Delta; x:A, \Theta}{(\nu x)(!x(y).P \mid Q) \vdash \Delta; \Theta} (\text{T}_{\text{cut}}?)$$

Reduction:

$$(x)(!x(y).P \mid \bar{x}(y).R) \rightarrow (x)(!x(y).P \mid (y)(R \mid P))$$

Main Results

Below $\cong_s$ denotes a process congruence capturing proof conversions (computational, structural, commuting):

Theorem (Cut Elimination)

If $P \vdash \Delta; \Theta$ then there is a process Q such that $P \cong_s Q$ and $Q \vdash \Delta; \Theta$ is derivable without using rules $(Tcut)$ and $(Tcut^?)$.

Theorem (Type Preservation)

If $P \vdash \Delta; \Theta$ and $P \rightarrow Q$ then $Q \vdash \Delta; \Theta$.

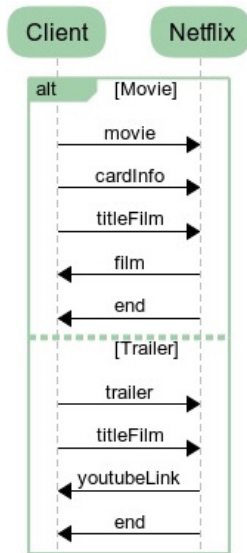
Write $live(P)$ whenever a prefix (non replicated) is at top-level:

Theorem (Progress)

If $P \vdash ; \Theta$ and $live(P)$ then there is Q such that $P \rightarrow Q$.

Also: strong normalization and confluence hold [Pérez et al.'12].

Example: A Simple Movie Server



Example: A Simple Movie Server

$$SBody(s) \triangleq s.case(s(title).s(card).\bar{s}(movie).s.\overline{close}, \\ s(title).\bar{s}(trailer).s.\overline{close})$$

$$Alice(s) \triangleq s.inr; \bar{s}("Odyssey").s(preview).s.close; 0$$

$$Bob(s) \triangleq s.inl; \bar{s}("Matrix").\bar{s}(bobscard).s(mpeg).s.close; 0$$

$$System_1 \triangleq (s)(SBody(s) \mid Alice(s))$$

Example: A Simple Movie Server

$$SBody(s) \triangleq s.\text{case}(s(\text{title}).s(\text{card}).\overline{s}(\text{movie}).s.\overline{\text{close}}, \\ s(\text{title}).\overline{s}(\text{trailer}).s.\overline{\text{close}})$$

$$Alice(s) \triangleq s.\text{inr}; \overline{s}(\text{"Odyssey"}).s(\text{preview}).s.\text{close}; 0$$

$$Bob(s) \triangleq s.\text{inl}; \overline{s}(\text{"Matrix"}).\overline{s}(\text{bobscard}).s(\text{mpeg}).s.\text{close}; 0$$

$$System_1 \triangleq (s)(SBody(s) \mid Alice(s))$$

Types

Recall that $A \multimap B$ and $\overline{A} \wp B$ are equivalent.

Assuming basic types `Title`, `CCard`, and `Mov` we have:

$$SBT \triangleq (\text{Title} \multimap \text{CCard} \multimap \text{Mov} \otimes 1) \ \& \ (\text{Title} \multimap \text{Mov} \otimes 1)$$

$$SBody(s) \vdash s : SBT ; \cdot$$

$$Alice(s) \vdash s : \overline{SBT} ; \cdot \quad Bob(s) \vdash s : \overline{SBT} ; \cdot$$

Movie Server, with Infinite Behavior

$$Movies(srv) \triangleq !srv(s).SBody(s)$$

$$Bob(s) \triangleq s.in1; \bar{s}("Matrix").\bar{s}(bobscard).s(mpeg).s.close; 0$$

$$SAlice(srv) \triangleq \overline{srv}(s).Alice(s)$$

$$SBob(srv) \triangleq \overline{srv}(s).Bob(s)$$

$$System_2 \triangleq (srv)(Movies(srv) \mid SAlice(srv) \mid SBob(srv))$$

Movie Server, with Infinite Behavior

$$Movies(srv) \triangleq !srv(s).SBody(s)$$

$$Bob(s) \triangleq s.in1; \bar{s}("Matrix").\bar{s}(bobscard).s(mpeg).s.close; 0$$

$$SAlice(srv) \triangleq \overline{srv}(s).Alice(s)$$

$$SBob(srv) \triangleq \overline{srv}(s).Bob(s)$$

$$System_2 \triangleq (srv)(Movies(srv) \mid SAlice(srv) \mid SBob(srv))$$

Types

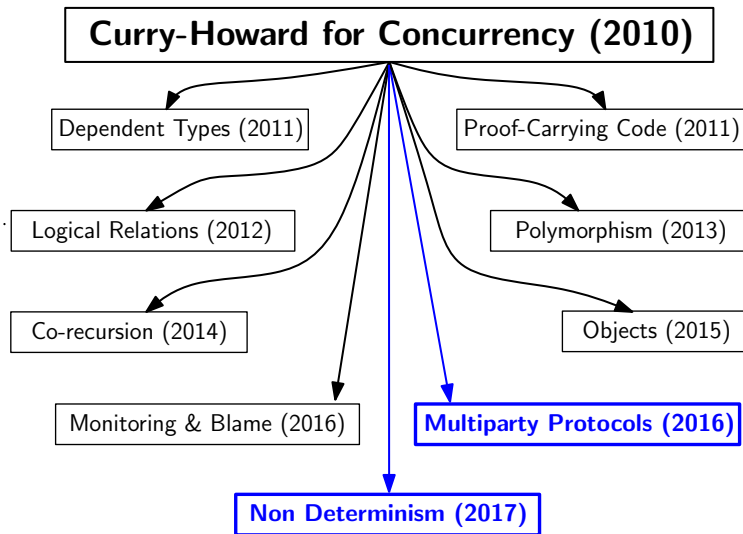
$$SAlice(srv) \vdash \cdot ; srv : \overline{SBT}$$

$$SBob(srv) \vdash \cdot ; srv : \overline{SBT}$$

$$SAlice(srv) \mid SBob(srv) \vdash srv : ?\overline{SBT}; \cdot$$

$$Movies(srv) \vdash srv : !SBT ; \cdot$$

Many Extensions!



Outline

Context

Logical Foundations for Concurrency

Curry-Howard correspondence for Concurrency

Example: Movie Server

Adding Non Determinism and Failure

Syntax and Semantics

Types

Properties

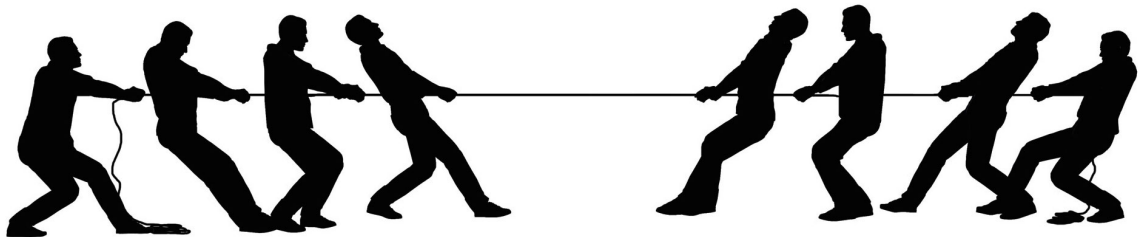
Examples

Extended Movie Server

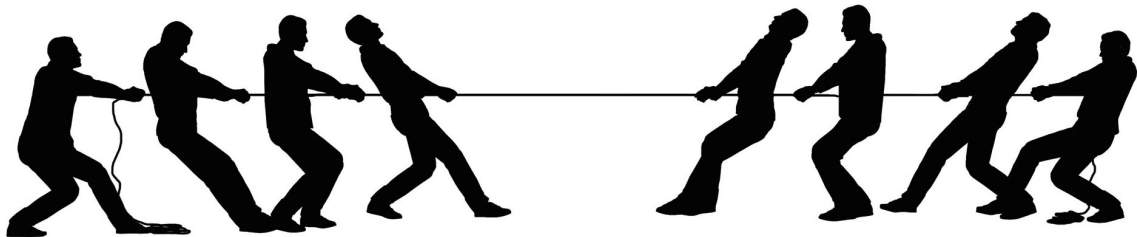
Encoding λ^{exc}

Concluding Remarks

Dealing with Long Known Tensions



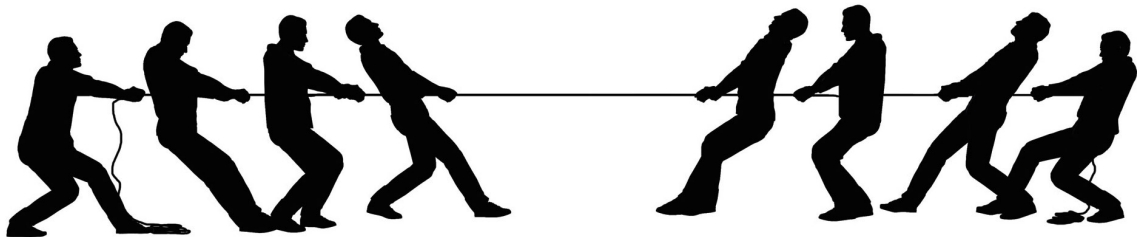
Dealing with Long Known Tensions



Linearity

Control effects (e.g. exceptions)

Dealing with Long Known Tensions

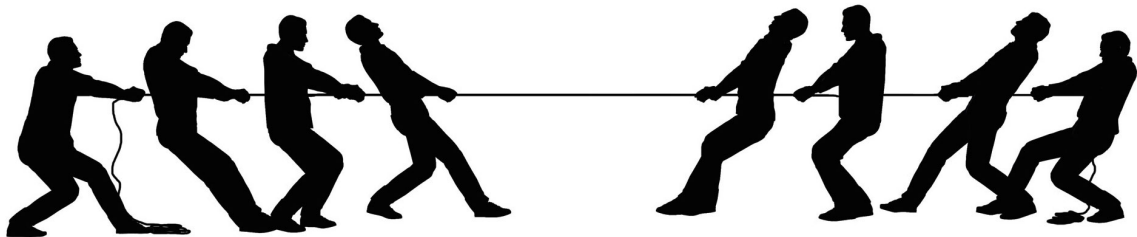


Linearity
Propositions as Types

Control effects (e.g. exceptions)
Non determinism

Dealing with Long Known Tensions

Concurrency, distribution, stateful resources, ...



Linearity
Propositions as Types

Control effects (e.g. exceptions)
Non determinism

Context

Much work on solving one or both tensions, including concurrent models and logic-based (Curry-Howard) approaches:

- Classical linear logic and call/cc [Griffin'90]
- Linear continuation passing (e.g. [Berdine et al.'01])
- Session types and interactional exceptions
[Carbone et al.'08; Capecchi et al.'10]
- Practical affinity in stateful settings [Tov & Pucella'10,11]
- Affine session types [Mostrous & Vasconcelos'14; Pfenning & Griffith'15]
- Mutually encoding sessions and effects [Orchard & Yoshida'16]

Context

Much work on solving one or both tensions, including concurrent models and logic-based (Curry-Howard) approaches:

- Classical linear logic and call/cc [Griffin'90]
- Linear continuation passing (e.g. [Berdine et al.'01])
- Session types and interactional exceptions
[Carbone et al.'08; Capecchi et al.'10]
- Practical affinity in stateful settings [Tov & Pucella'10,11]
- Affine session types [Mostrous & Vasconcelos'14; Pfenning & Griffith'15]
- Mutually encoding sessions and effects [Orchard & Yoshida'16]

Still, no programming model uniformly supports concurrency, linearity, non-determinism, and control effects while

- following logical principles in a Curry-Howard style
- enforcing key properties (progress, SN, confluence)

Our Contributions (1/2)

An extension of the interpretation of classical linear logic as session types, covering **non determinism** and **failure**

Our Contributions (1/2)

An extension of the interpretation of classical linear logic as session types, covering **non determinism** and **failure**

Key Features

- At the **type level**, two new modalities:
 - Type $\& A$ for sessions that **may produce** a behavior of type A
 - Type $\oplus A$ for sessions that **may consume** a behavior of type A
- At the **process level**:
 - Explicit prefixes: $x.\overline{\text{some}}; P$ $x.\overline{\text{none}}; P$ $x.\text{some}_{\overline{w}}; P$
 - Internal non-determinism: $P \oplus Q$

Our Contributions (2/2)

Main results

1. Cut elimination for CLL with new modalities
2. Type preservation, global progress (deadlock-freedom), strong normalization, confluence
3. Compatibility of standard (non confluent) and internal (confluent) non-determinism
4. Type-preserving encodability of λ^{exc} :
 λ -calculus + threads, exceptions, sessions & non-determinism

Process Model: Syntax

$P, Q ::= \overline{x}(y).P \mid x(y).P \mid !x(y).P$	[output and (replicated) input]
$\mid P \mid Q \mid (\nu y)P \mid 0$	[parallel, restriction, inaction]
$\mid x.\text{case}(P, Q)$	[branching]
$\mid x.\text{inr}; P \mid x.\text{inl}; P$	[right and left selection]
$\mid x.\overline{\text{close}} \mid x.\text{close}; P$	[session termination]
$\mid [x \leftrightarrow y]$	[channel forwarder]

Process Model: Syntax

$P, Q ::= \overline{x}(y).P \mid x(y).P \mid !x(y).P$	[output and (replicated) input]
$\mid P \mid Q \mid (\nu y)P \mid 0$	[parallel, restriction, inaction]
$\mid x.\text{case}(P, Q)$	[branching]
$\mid x.\text{inr}; P \mid x.\text{inl}; P$	[right and left selection]
$\mid x.\overline{\text{close}} \mid x.\text{close}; P$	[session termination]
$\mid [x \leftrightarrow y]$	[channel forwarder]
$\mid x.\overline{\text{some}}; P \mid x.\overline{\text{none}}; P$	Produce non-det behavior
$\mid x.\text{some}_{w_1 \dots w_n}; P$	Consume non-det behavior
$\mid P \oplus Q$	Internal non-determinism

Process Model: Reduction Semantics [Excerpt]

Reduction ($\rightarrow$) plus structural congruence ($\equiv$):

$$\begin{aligned}\bar{x}(y).Q \mid x(y).P &\rightarrow (y)(Q \mid P) \\ \bar{x}(y).Q \mid !x(y).P &\rightarrow (y)(Q \mid P) \mid !x(y).P \\ x.\text{inl}; P \mid x.\text{case}(Q, R) &\rightarrow P \mid Q \\ x.\overline{\text{close}} \mid x.\text{close}; P &\rightarrow P \\ (x)([x \leftrightarrow y] \mid P) &\rightarrow P\{y/x\} \\ P \equiv P', P' \rightarrow Q', Q' \equiv Q &\Rightarrow P \rightarrow Q\end{aligned}$$

Process Model: Reduction Semantics [Excerpt]

Reduction ($\rightarrow$) plus structural congruence ($\equiv$):

$$\begin{aligned}\bar{x}(y).Q \mid x(y).P &\rightarrow (y)(Q \mid P) \\ \bar{x}(y).Q \mid !x(y).P &\rightarrow (y)(Q \mid P) \mid !x(y).P \\ x.\text{inl}; P \mid x.\text{case}(Q, R) &\rightarrow P \mid Q \\ x.\overline{\text{close}} \mid x.\text{close}; P &\rightarrow P \\ (x)([x \leftrightarrow y] \mid P) &\rightarrow P\{y/x\} \\ P \equiv P', P' \rightarrow Q', Q' \equiv Q &\Rightarrow P \rightarrow Q \\ \\ x.\overline{\text{some}}; P \mid x.\text{some}_{w_1 \dots w_n}; Q &\rightarrow P \mid Q \\ x.\overline{\text{none}} \mid x.\text{some}_{w_1 \dots w_n}; Q &\rightarrow w_1.\overline{\text{none}} \mid \dots \mid w_n.\overline{\text{none}} \\ Q \rightarrow Q' \Rightarrow P \oplus Q &\rightarrow P \oplus Q' \\ P \oplus Q &\equiv Q \oplus P \\ (x)(P \mid (Q \oplus R)) &\equiv (x)(P \mid Q) \oplus (x)(P \mid R)\end{aligned}$$

Session Types with Non Determinism

- A Curry-Howard correspondence for concurrency enhancing prior works [Caires & Pfenning'10; Wadler'12]:

linear logic propositions	$\leftrightarrow$	session types
sequent proofs	$\leftrightarrow$	π -calculus processes
cut elimination	$\leftrightarrow$	process communication

- Type judgments describe the protocols implemented by a process, linear and unrestricted
- Typing rules correspond to the sequent calculus for CLL with mix, plus **new modalities** $\&A$ and $\oplus A$, related by duality
- Key principles:
 - **As before**: Identity as forwarding; cut as parallel + restriction
 - **New**: Monadic approach to non-det via the dual modalities

Types and Duality

Types are assigned to names and describe their session behavior:

$x : A \otimes B$ [Output name of type A , continue as B]

$x : A \wp B$ [Input name of type A , continue as B]

$x : !A$ [Persistent offer of A]

$x : ?A$ [Request of a persistent behavior A]

$x : A \& B$ [Offer *both* A and B]

$x : A \oplus B$ [Select *either* A or B]

$x : 1$ [Terminated interaction]

$x : \perp$ [Terminated interaction]

Duality of A , written $\overline{A}$, is standard

Types and Duality

Types are assigned to names and describe their session behavior:

$x : A \otimes B$	[Output name of type A , continue as B]
$x : A \wp B$	[Input name of type A , continue as B]
$x : !A$	[Persistent offer of A]
$x : ?A$	[Request of a persistent behavior A]
$x : A \& B$	[Offer <i>both</i> A and B]
$x : A \oplus B$	[Select <i>either</i> A or B]
$x : 1$	[Terminated interaction]
$x : \perp$	[Terminated interaction]
$x : \& A$	Produce some non-det behavior of type A on x
$x : \oplus A$	Consume any non-det behavior of type A on x

Duality of A , written $\overline{A}$, is standard, with $\overline{\oplus A} = \& \overline{A}$ and $\overline{\& A} = \oplus \overline{A}$.

Non Determinism & Failure: Typing Rules

$$\frac{\vdash \Delta, A; \Theta}{\vdash \Delta, \&A; \Theta} (\top \&_d^x)$$

$$\frac{}{\vdash \&A; \Theta} (\top \&^x)$$

$$\frac{\vdash \&\Delta, A; \Theta}{\vdash \&\Delta, \oplus A; \Theta} (\top \oplus_{\vec{w}}^x)$$

$$\frac{\vdash \&\Delta; \Theta \quad \vdash \&\Delta; \Theta}{\vdash \&\Delta; \Theta} (\top \&)$$

Non Determinism & Failure: Typing Rules

$$\frac{P \vdash \Delta, x:A; \Theta}{x.\text{some}; P \vdash \Delta, x:\&A; \Theta} (\text{T}\&_d^x)$$

$$\frac{}{x.\text{none} \vdash x:\&A; \Theta} (\text{T}\&^x)$$

$$\frac{P \vdash \overline{w}:\&\Delta, x:A; \Theta}{x.\text{some}_{\overline{w}}; P \vdash \overline{w}:\&\Delta, x:\oplus A; \Theta} (\text{T}\oplus_{\overline{w}}^x)$$

$$\frac{P \vdash \&\Delta; \Theta \quad Q \vdash \&\Delta; \Theta}{P \oplus Q \vdash \&\Delta; \Theta} (\text{T}\&)$$

Non Determinism & Failure: Semantics (1/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{\frac{P \vdash \overline{w}:\&\Delta, x:A}{x.\text{some}_{\overline{w}}; P \vdash \overline{w}:\&\Delta, x:\oplus A} \quad \frac{Q \vdash \Delta', x:\overline{A}}{x.\overline{\text{some}}; Q \vdash \Delta', x:\&\overline{A}}}{(x)(x.\text{some}_{\overline{w}}; P \mid x.\overline{\text{some}}; Q) \vdash \overline{w}:\&\Delta, \Delta'}$$

Non Determinism & Failure: Semantics (1/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{\frac{P \vdash \bar{w}:\&\Delta, x:A}{x.\text{some}_{\bar{w}}; P \vdash \bar{w}:\&\Delta, x:\oplus A} \quad \frac{Q \vdash \Delta', x:\bar{A}}{x.\overline{\text{some}}; Q \vdash \Delta', x:\&\bar{A}}}{(x)(x.\text{some}_{\bar{w}}; P \mid x.\overline{\text{some}}; Q) \vdash \bar{w}:\&\Delta, \Delta'} \rightarrow \frac{P \vdash \bar{w}:\&\Delta, x:A \quad Q \vdash \Delta', x:\bar{A}}{(x)(P \mid Q) \vdash \bar{w}:\&\Delta, \Delta'}$$

Non Determinism & Failure: Semantics (2/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{\frac{P \vdash \overline{w}:\&\Delta, x:A}{x.\text{some}_{\overline{w}}; P \vdash \overline{w}:\&\Delta, x:\oplus A} \quad \frac{}{x.\overline{\text{none}} \vdash x:\&\overline{A}}}{(x)(x.\text{some}_{\overline{w}}; P \mid x.\overline{\text{none}}) \vdash \overline{w}:\&\Delta}$$

Non Determinism & Failure: Semantics (2/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{\frac{P \vdash \overline{w}:\&\Delta, x:A}{x.\text{some}_{\overline{w}}; P \vdash \overline{w}:\&\Delta, x:\oplus A} \quad \frac{}{x.\overline{\text{none}} \vdash x:\&\overline{A}}}{(x)(x.\text{some}_{\overline{w}}; P \mid x.\overline{\text{none}}) \vdash \overline{w}:\&\Delta} \rightarrow \frac{}{w_1.\overline{\text{none}} \mid \cdots \mid w_n.\overline{\text{none}} \vdash \overline{w}:\&\Delta}$$

Non Determinism & Failure: Semantics (3/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{P \vdash \&\Delta, x:A \quad \frac{Q \vdash \&\Delta', x:\&\bar{A} \quad R \vdash \&\Delta', x:\&\bar{A}}{Q \oplus R \vdash \&\Delta', x:\&\bar{A}}}{(x)(P \mid (Q \oplus R)) \vdash \&\Delta, \&\Delta'}$$

Non Determinism & Failure: Semantics (3/3)

[Omitting the exponential context Θ , for simplicity]

$$\frac{\frac{P \vdash \&\Delta, x:A}{(x)(P \mid (Q \oplus R)) \vdash \&\Delta, \&\Delta'} \quad \frac{\frac{Q \vdash \&\Delta', x:\&\bar{A} \quad R \vdash \&\Delta', x:\&\bar{A}}{Q \oplus R \vdash \&\Delta', x:\&\bar{A}}}{(x)(P \mid (Q \oplus R)) \vdash \&\Delta, \&\Delta'} \rightarrow$$
$$\frac{\frac{P \vdash \&\Delta, x:A \quad Q \vdash \&\Delta', x:\&\bar{A}}{(x)(P \mid Q) \vdash \&\Delta, \&\Delta'} \quad \frac{P \vdash \&\Delta, x:A \quad R \vdash \&\Delta', x:\&\bar{A}}{(x)(P \mid R) \vdash \&\Delta, \&\Delta'}}{(x)(P \mid Q) \oplus (x)(P \mid R) \vdash \&\Delta, \&\Delta'}$$

- A congruence axiom, rather than a reduction rule
Non-collapsing: all non-deterministic branches are kept open
- Natural distribution law of parallel over internal non-det choice

Main Results (1/2)

Write $\cong_s$ to denote a process congruence capturing proof conversions (computational, structural, commuting):

Theorem (Cut Elimination)

If $P \vdash \Delta; \Theta$ then there is a process Q such that $P \cong_s Q$ and $Q \vdash \Delta; \Theta$ is derivable without using rules $(Tcut)$ and $(Tcut^?)$.

Theorem (Type Preservation)

If $P \vdash \Delta; \Theta$ and $P \rightarrow Q$ then $Q \vdash \Delta; \Theta$.

Write $live(P)$ whenever a prefix (non replicated) is at top-level:

Theorem (Progress)

If $P \vdash ; \Theta$ and $live(P)$ then there is Q such that $P \rightarrow Q$.

Also: strong normalization and confluence hold [Pérez et al.'12].

Main Results (2/2)

Write $P \rightarrow_c Q$ (and $P \Rightarrow_c Q$) to denote reduction extended with collapsing (non-confluent) rules $P \oplus Q \rightarrow P$ and $P \oplus Q \rightarrow Q$.

We say P is **prime** if $P \not\equiv (Q \oplus R)$ with non-trivial Q and R .

Theorem (Postponing)

Let $P \vdash \Delta; \Theta$. We have

1. If $P \Rightarrow P_1 \oplus \dots \oplus P_n \not\rightarrow$ with P_i prime for all i , then $P \Rightarrow_c P_i$ for all $0 < i \leq n$.
2. Let $\mathcal{C} = \{ P_i \mid P \Rightarrow_c P_i \not\rightarrow_c \text{ and } P_i \text{ prime} \}$. Then $\mathcal{C}$ is finite up to $\equiv$, with $\#\mathcal{C} = n$, and $P \Rightarrow P_1 \oplus \dots \oplus P_n \rightarrow_c P_i$ ($0 < i \leq n$).

Outline

Context

Logical Foundations for Concurrency

Curry-Howard correspondence for Concurrency

Example: Movie Server

Adding Non Determinism and Failure

Syntax and Semantics

Types

Properties

Examples

Extended Movie Server

Encoding λ^{exc}

Concluding Remarks

A Simple Movie Server

We showcase our typed model with a simple movie server process, incrementally featuring:

- Basic session communication and deterministic choices
- Non deterministic choices
- Failure

A Simple Movie Server

$$SBody(s) \triangleq s.\text{case}(s(title).s(card).\overline{s}(movie).s.\overline{\text{close}}, \\ s(title).\overline{s}(trailer).s.\overline{\text{close}})$$

$$Alice(s) \triangleq s.\text{inr}; \overline{s}("Odyssey").s(preview).s.\text{close}; 0$$

$$Bob(s) \triangleq s.\text{inl}; \overline{s}("Matrix").\overline{s}(bobscard).s(mpeg).s.\text{close}; 0$$

$$System_1 \triangleq (s)(SBody(s) \mid Alice(s))$$

Types

Recall that $A \multimap B$ and $\overline{A} \wp B$ are equivalent.

Assuming basic types `Title`, `CCard`, and `Mov` we have:

$$SBT \triangleq (\text{Title} \multimap \text{CCard} \multimap \text{Mov} \otimes 1) \ \& \ (\text{Title} \multimap \text{Mov} \otimes 1)$$

$$SBody(s) \vdash s : SBT ; \cdot$$

$$Alice(s) \vdash s : \overline{SBT} ; \cdot \quad Bob(s) \vdash s : \overline{SBT} ; \cdot$$

Movie Server, with Infinite Behavior

$$Movies(srv) \triangleq !srv(s).SBody(s)$$

$$Bob(s) \triangleq s.in1; \bar{s}("Matrix").\bar{s}(bobscard).s(mpeg).s.close; 0$$

$$SAlice(srv) \triangleq \overline{srv}(s).Alice(s)$$

$$SBob(srv) \triangleq \overline{srv}(s).Bob(s)$$

$$System_2 \triangleq (srv)(Movies(srv) \mid SAlice(srv) \mid SBob(srv))$$

Types

$$SAlice(srv) \vdash \cdot ; srv : \overline{SBT}$$

$$SBob(srv) \vdash \cdot ; srv : \overline{SBT}$$

$$SAlice(srv) \mid SBob(srv) \vdash srv : ?\overline{SBT}; \cdot$$

$$Movies(srv) \vdash srv : !SBT ; \cdot$$

Movie Server, with Non Determinism

A non-deterministic client:

$$Randy(s) \triangleq s.\overline{\text{some}}; Alice(s) \oplus s.\overline{\text{some}}; Bob(s)$$

$$USystem \triangleq (s)(s.\text{some}_{\emptyset}; SBody(s) \mid Randy(s))$$

We derive $USystem \vdash \cdot ; \cdot$

Movie Server, with Non Determinism

A non-deterministic client:

$$Randy(s) \triangleq s.\overline{\text{some}}; Alice(s) \oplus s.\overline{\text{some}}; Bob(s)$$

$$USystem \triangleq (s)(s.\text{some}_\emptyset; SBody(s) \mid Randy(s))$$

We derive $USystem \vdash \cdot ; \cdot$ as follows. First, the non-det client:

$$\frac{\frac{Alice(s) \vdash s : \overline{SBT} ; \cdot}{s.\overline{\text{some}}; Alice(s) \vdash s : \&\overline{SBT} ; \cdot} \quad \frac{Bob(s) \vdash s : \overline{SBT} ; \cdot}{s.\overline{\text{some}}; Bob(s) \vdash s : \&\overline{SBT} ; \cdot}}{s.\overline{\text{some}}; Alice(s) \oplus s.\overline{\text{some}}; Bob(s) \vdash s : \&\overline{SBT} ; \cdot}$$

Movie Server, with Non Determinism

A non-deterministic client:

$$Randy(s) \triangleq s.\overline{\text{some}}; Alice(s) \oplus s.\overline{\text{some}}; Bob(s)$$

$$USystem \triangleq (s)(s.\text{some}_\emptyset; SBody(s) \mid Randy(s))$$

We derive $USystem \vdash \cdot ; \cdot$ as follows. First, the non-det client:

$$\frac{\frac{Alice(s) \vdash s : \overline{SBT} ; \cdot}{s.\overline{\text{some}}; Alice(s) \vdash s : \&\overline{SBT} ; \cdot} \quad \frac{Bob(s) \vdash s : \overline{SBT} ; \cdot}{s.\overline{\text{some}}; Bob(s) \vdash s : \&\overline{SBT} ; \cdot}}{s.\overline{\text{some}}; Alice(s) \oplus s.\overline{\text{some}}; Bob(s) \vdash s : \&\overline{SBT} ; \cdot}$$

Then we compose the non-det client with the (adapted) server:

$$\frac{\frac{SBody(s) \vdash s : SBT ; \cdot}{s.\text{some}_\emptyset; SBody(s) \vdash s : \oplus SBT ; \cdot} \quad \frac{Randy(s) \vdash s : \&\overline{SBT} ; \cdot}{USystem \vdash \cdot ; \cdot}}$$

Movie Server, with NonDet and a Boolean log

A server with a log, with type $B = 1 \oplus 1$:

$$SBodyL(s) \triangleq s.\text{case}(s(title).\bar{s}(trailer).s.\overline{\text{close}} \mid l.\text{inl}; l.\overline{\text{close}}, \\ s(tit).s(card).\bar{s}(mov).s.\overline{\text{close}} \mid l.\text{inr}; l.\overline{\text{close}})$$

We have $SBodyL(s) \vdash s:SBT, l:B ; \cdot$.

But this can't be composed with client $Randy(s) \vdash s : \&\overline{SBT} ; \cdot$.

Movie Server, with NonDet and a Boolean log

A server with a log, with type $B = 1 \oplus 1$:

$$SBodyL(s) \triangleq s.\text{case}(s(\text{title}).\overline{s}(\text{trailer}).s.\overline{\text{close}} \mid l.\text{inl}; l.\overline{\text{close}}, \\ s(\text{tit}).s(\text{card}).\overline{s}(\text{mov}).s.\overline{\text{close}} \mid l.\text{inr}; l.\overline{\text{close}})$$

We have $SBodyL(s) \vdash s:SBT, l:B ; \cdot$

But this can't be composed with client $Randy(s) \vdash s : \&\overline{SBT} ; \cdot$

Rather, we could have:

$$s.\text{some}_l; l.\overline{\text{some}}; SBodyL(s) \vdash s: \oplus SBT, l:\&B ; \cdot$$

and derive:

$$ULSystem \triangleq (s)(s.\text{some}; l.\overline{\text{some}}; SBodyL(s) \mid Randy(s)) \\ ULSystem \vdash l:\&B ; \cdot$$

Movie Server, with Failure

A faulty client:

$$Buzz(s) \triangleq s.\overline{\text{some}}; Bob(s) \oplus s.\overline{\text{none}}$$

Movie Server, with Failure

A faulty client:

$$Buzz(s) \triangleq s.\overline{\text{some}}; Bob(s) \oplus s.\overline{\text{none}}$$

It has the same type as the non-det client $Randy(s)$:

$$Buzz(s) \vdash s : \&\overline{SBT} ; \cdot$$

and so we can compose it with the server with log:

$$(s)(SBodyL(s) \mid Buzz(s)) \vdash l:\&B ; \cdot$$

Movie Server, with Failure

A faulty client:

$$Buzz(s) \triangleq s.\overline{\text{some}}; Bob(s) \oplus s.\overline{\text{none}}$$

It has the same type as the non-det client $Randy(s)$:

$$Buzz(s) \vdash s : \&\overline{SBT} ; \cdot$$

and so we can compose it with the server with log:

$$(s)(SBodyL(s) \mid Buzz(s)) \vdash l:\&B ; \cdot$$

Failure of sub-computations propagates inside the monad $\&-$.

We have the reduction sequence:

$$(s)(SBodyL(s) \mid Buzz(s)) \Rightarrow (l.\overline{\text{none}} \oplus l.\text{inl}; l.\overline{\text{close}})$$

The system thus either aborts or chooses $l.\text{inl}$ on the log.

λ^{exc} : Functions, Sessions, Threads & Non-det

An extended case study for the expressiveness of our type system.

Expressions

$$\begin{aligned} v &::= x \mid * \mid \lambda z. e \mid \langle\langle v \rangle\rangle \\ e &::= v \mid (f \ x) \mid \text{LET } a = e_1 \text{ IN } e_2 \\ &\quad \mid \text{TRY } e_1 \text{ CATCH } z. e_2 \mid \text{THROW } z \\ &\quad \mid \text{LIFT } e \mid \text{SOME } !z; e \mid \text{SOME } ?z; e \mid \text{NONE } !z; e \mid e_1 \oplus e_2 \\ &\quad \mid \text{FORK } c. e \mid \text{SEND}(c, e_1); e_2 \mid \text{RECV}(c, z); e \\ &\quad \mid \text{CLOSE } !c; e \mid \text{CLOSE } ?c \end{aligned}$$

Types

Expressions are **effectful** (if they can raise an exception) or **pure**

$$\begin{aligned} T &::= \text{unit} \mid A \xrightarrow{T} B \mid A \xrightarrow{0} B \\ &\quad \mid !T. T' \mid ?T. T' \mid \text{end}_! \mid \text{end}_? \mid \oplus T \mid \& T \end{aligned}$$

$A \xrightarrow{T} B$ types functions that may raise an exception of type T .

$A \xrightarrow{0} B$ is the type of pure functions.

Types for λ^{exc}

- A type-and-effect system: effects represent the type of the exception that can be raised
- Judgments are then the form

$$D \vdash^U e : T$$

Under environment D , expression e has return type T , while the effect type U is 0 (e is pure) or T' (the type of exceptions)

Sample Typing Rules for λ^{exc}

$$\text{LET1} \quad \frac{D_1 \vdash^T e_1 : A \quad \oplus D_2, a:A \vdash^T e_2 : B}{D_1, \oplus D_2 \vdash^T \text{LET } a = e_1 \text{ IN } e_2 : B}$$

$$\text{LET2} \quad \frac{D_1 \vdash^0 e_1 : A \quad D_2, a:A \vdash^0 e_2 : B}{D_1, D_2 \vdash^0 \text{LET } a = e_1 \text{ IN } e_2 : B}$$

$$\text{LET3} \quad \frac{D_1 \vdash^0 e_1 : A \quad D_2, a:A \vdash^T e_2 : B}{D_1, D_2 \vdash^T \text{LET } a = e_1 \text{ IN } e_2 : B}$$

$$\text{TRY} \quad \frac{D_1 \vdash^U e_1 : T \quad x : U \vdash^0 e_2 : T}{D_1 \vdash^0 \text{TRY } e_1 \text{ CATCH } x.e_2 : T}$$

$$\text{THROW} \quad \frac{D \vdash^0 z : T}{D \vdash^T \text{THROW } z : U}$$

λ^{exc} into Typed Processes: Examples (1/2)

In $[e]_{y,x}$, name y is a **non-deterministic continuation** where values may be produced; name x denotes the **enclosing handler**.

- Variables

$$[z]_y = y\langle z \rangle.y.\overline{\text{close}}$$

$$[z]_{y,x} = y.\overline{\text{some}}; y\langle z \rangle.y\langle x \rangle.y.\overline{\text{close}}$$

λ^{exc} into Typed Processes: Examples (1/2)

In $[e]_{y,x}$, name y is a **non-deterministic continuation** where values may be produced; name x denotes the **enclosing handler**.

- Variables

$$\begin{aligned}[z]_y &= y\langle z \rangle.y.\overline{\text{close}} \\ [z]_{y,x} &= y.\overline{\text{some}}; y\langle z \rangle.y\langle x \rangle.y.\overline{\text{close}}\end{aligned}$$

- Let expressions

$$\begin{aligned}[\text{LET } a = e_1 \text{ IN } e_2]_y &= (q)([e_1]_q \mid q(a).q.\text{close}; [e_2]_y) \\ [\text{LET } a = e_1 \text{ IN } e_2]_{y,x} &= (q)([e_1]_{q,x} \mid \\ &\quad q.\overline{\text{some}}_D; q(a).q(s).q.\text{close}; [e_2]_{y,s})\end{aligned}$$

λ^{exc} into Typed Processes: Examples (2/2)

- Try-catch

$$\begin{aligned}
 [\text{TRY } e_1 \text{ CATCH } z. e_2]_y = & \\
 (j) \Big((k) \Big([e_1]_{k,j} \mid & \\
 \underbrace{k.\text{some}_\emptyset; k(u).k(z).k.\text{close}; z.\text{inl}; z\langle u \rangle.z.\overline{\text{close}}}_{(I)} \mid & \\
 \underbrace{j.\text{case}(j(u).j.\text{close}; y\langle u \rangle.y.\overline{\text{close}} , j(z).j.\text{close}; [e_2]_y)}_{(II)} \Big) &
 \end{aligned}$$

where (I) deals with normal behavior,
while (II) handles both normal and exceptional behaviors.

- Throw:

$$[\text{THROW } z]_{y,x} = y.\overline{\text{none}} \mid x.\text{inr}; x\langle z \rangle.x.\bar{0}$$

Additional Results

(Joint work with Joseph Paulus and Daniele Nantes)

- $\lambda_{\oplus}^{\zeta}$: a resource λ -calculus with non-determinism, bags, and an explicit term denoting failures.
- The syntax and semantics of $\lambda_{\oplus}^{\zeta}$ are based on existing resource λ -calculi
 - ▶ Boudol and Laneve (2000)
 - ▶ Pagani and Ronchi della Rocca (2010)while adopting non-collapsing determinism.
- An **intersection type system** to “count” resources in bags.
- We have encoded $\lambda_{\oplus}^{\zeta}$ into the session π -calculus just presented.
 - ▶ The encoding considers **well formed** expressions in $\lambda_{\oplus}^{\zeta}$: well-typed expressions *plus* expressions that lead to failure.
 - ▶ The encoding preserves typing and satisfies operational correspondences.

Outline

Context

Logical Foundations for Concurrency

Curry-Howard correspondence for Concurrency

Example: Movie Server

Adding Non Determinism and Failure

Syntax and Semantics

Types

Properties

Examples

Extended Movie Server

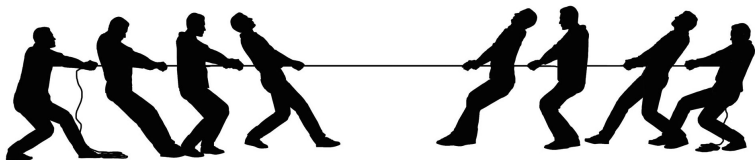
Encoding λ^{exc}

Concluding Remarks

Concluding Remarks

- A Curry-Howard correspondence for concurrency, exploiting linear logic and name-passing processes under session types
- Strong correctness properties on protocol conformance follow from logical principles (e.g., cut elimination)
- Excellent basis for extensions, via developments in logic

Concluding Remarks



- Tackling stateful concurrent programs with linearity, based on a Curry-Howard interpretation of extended CLL as session types
- Our type system supports non-determinism and failure within concurrent programs by exploiting new modalities $\&A$ and $\oplus A$
- Strong correctness properties (e.g. global progress); non-determinism compatible with process calculi approaches

Logical Foundations for Concurrency: Non-Determinism and Failures

Jorge A. Pérez

University of Groningen, The Netherlands

j.a.perez [[at]] rug.nl

ECI 2026

(Version of 31st July 2026)